CHAPTER 05

API design and integration patterns

For Business and Technical Analysts in international commercial banking

Antonio Piemontese

October 2026



CHAPTER 05

Learning objectives

01 **Specify API contracts**

Define resources, operations and errors that callers code against.

02 **Make retries safe**

Apply idempotency keys, timeouts and unambiguous status codes.

03 **Evolve contracts safely**

Plan additive change, versioning and an explicit sunset path.

04 **Select integration patterns**

Match direct, brokered, batch and streaming styles to the work.

API design and integration patterns

API design

Defines what each operation means and returns.

Names resources, errors and deadlines.

Is the contract callers write code against.

Integration patterns

Moves requests and messages between systems.

Sets delivery, ordering and retry behaviour.

Is the transport that contract travels on.

Patterns move messages. The API contract defines what they mean.

API DESIGN & INTEGRATION PATTERNS

Two complementary decisions: contract and transport



API DESIGN

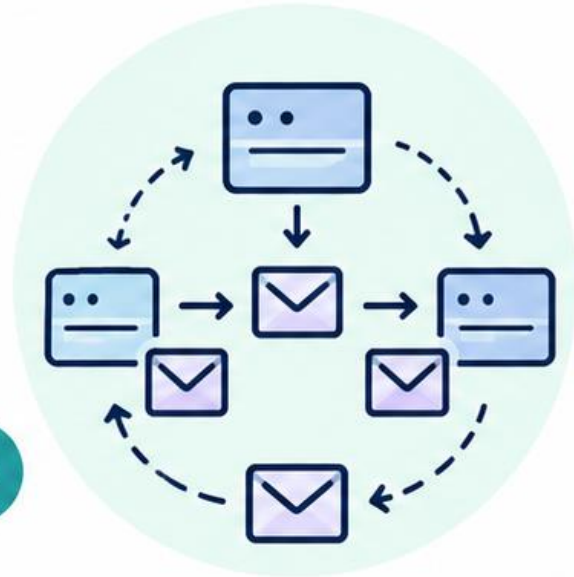
THE CONTRACT

What do messages mean?

Operations & results
Defines what each operation means and returns.

Resources, errors & deadlines
Makes expectations explicit.

Caller expectations
The contract callers write code against.



INTEGRATION PATTERNS

THE TRANSPORT

How do messages move?

Requests & messages
Moves information between systems.

Delivery & ordering
Defines delivery and sequence behaviour.

Retries
Defines how failed delivery is retried.

Patterns **move messages.**
The API contract defines **what they mean.**

The anatomy of an API contract

An API contract fixes resources, operation semantics and the errors a caller must handle.

Callers see /loan-applications and its promises, not the workflow behind it.

Resource model

Nouns with identity: /loan-applications/L-1042.

Operation semantics

Safe reads, idempotent updates, guarded creates.

Error contract

Machine-readable errors naming the next step.

Service levels

Deadlines and limits that callers plan on.

HYPOTHETICAL SCENARIO EXTENSION

Banking pilot: a partner submission API

New channel need

A partner portal now wants to submit loan applications and track their status through a public API of the pilot.

Continuing boundaries

Intake and workflow stay inside.

Existing credit authority remains.

Events keep feeding notifications.

Decision to evaluate	Can one public contract serve partners without exposing internal workflow?

The starting point (what the slide does not repeat)

In the previous chapters we built a teaching case: the "pilot", that is, a **Loan Application System (LAS)** — the system the bank uses to receive and process loan applications — running in a single country as a pilot project. So far, the only users of this system are the bank's own **internal channels**: the bank's website and mobile app. "Channel" here simply means the entry door through which applications arrive (branch, website, app…).

What is new: the partner

The scenario invents a new requirement (that is why the label says *hypothetical scenario extension*): an **external partner** shows up — think of a loan comparison portal, or a network of credit brokers. The partner wants two things:

1. **submit** loan applications on behalf of its users (the customer fills in the form on the partner's website, and the partner's system forwards the application to the bank);
2. **track their status** ("is it under assessment? has it been recorded?") so it can show it to the customer.

The partner, however, is an external company: it cannot use the bank's internal APIs. So the pilot needs a **public API**, that is, an interface exposed outside the bank's perimeter — "public" in the sense of *reachable by an external party under contract*, not "open to anyone".

"Continuing boundaries": what does NOT change

The right-hand column pins down the fixed points. The "boundaries" are the lines of responsibility drawn in the previous chapters, and the scenario deliberately keeps them in place:

- *Intake and workflow stay inside* — receiving the application (intake) and processing it (workflow) remain inside the pilot. The partner hands the application over, full stop: it neither sees nor drives the internal process.
- *Existing credit authority remains* — the decision on the loan still belongs to the Credit Decision Service. The new API does not move credit authority one inch: the partner cannot influence the assessment.
- *Events keep feeding notifications* — internally, the chapter 4 mechanism (the ApplicationRecorded event feeding notifications) continues unchanged.

Why insist on what does not change? It is the teaching technique of the whole course: **isolate a single new variable**. In chapter 4 the variable was "a team wants to release independently"; here it is "an external party wants a public contract". Everything else is frozen, so the discussion stays on the point.

The "Decision to evaluate"

"Can one public contract serve partners without exposing internal workflow?" — there are two questions inside it:

1. **One contract**: do we build a single API v1 that serves this partner (and any future ones), or a bespoke API for each channel? A single contract is easier to govern, but it has to be general enough.
2. **Without exposing internal workflow**: the temptation is to let the partner see the internal processing states ("queued at service X", "waiting for the bureau…"). That would be a mistake: every internal detail we expose becomes a constraint — if tomorrow we reorganise the process, we break the partner. The API must expose only the contract's promises: the submission, the identifier L-1042, a few public statuses (e.g. *Pending assessment*), the errors.

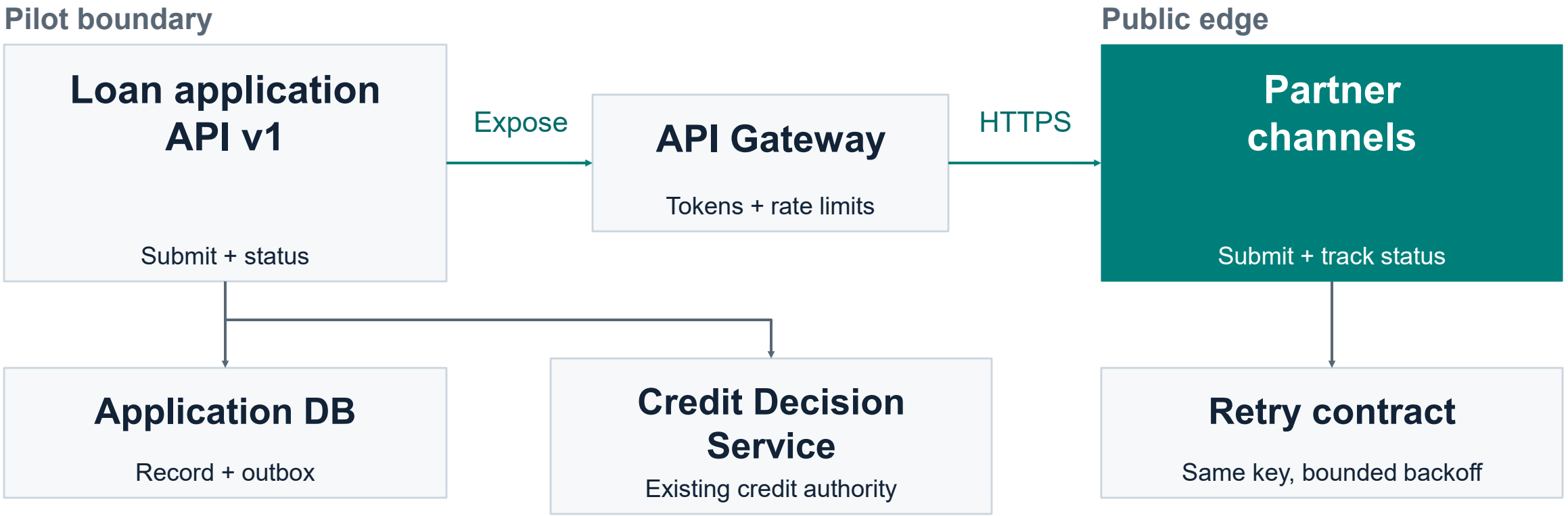
The full journey, to make it concrete

The customer fills in the form on the partner's portal → the partner's system calls `POST /loan-applications` on the bank's public API → it receives `201` with the identifier `L-1042` → it periodically calls `GET /loan-applications/L-1042/status` → it shows the customer "application under assessment" → the bank decides internally, at its own pace → the status changes and the partner sees it at the next poll.

Why this scenario carries the whole chapter: the partner is external and calls over the Internet, so there will be timeouts and retransmissions → we need idempotency (slides 7, 10, 12); it is external, so we need the gateway with tokens and rate limits (slides 6, 14); we cannot "redeploy" the partner ourselves when we change the API → we need versioning (slide 11); it runs reconciliations → we need pagination (slide 13). The decision is then formalised in ADR-005 (slide 17).

HYPOTHETICAL API DESIGN

Proposed API surface



Channels reach one contract through the gateway. Data, credit authority and events are unchanged.

OPERATION SEMANTICS

Safe, idempotent and unsafe operations

Class	Example	Retry behaviour
Safe	GET …/L-1042/status	Repeat freely: reading changes nothing
Idempotent	PUT …/L-1042/contact	Same request twice leaves one result
Unsafe	POST /loan-applications	Needs an idempotency key to retry

Retry rules follow the operation class, not the transport that carries it.

ERROR CONTRACTS

Errors that tell the caller what to do

Change the request

400 invalid payload: fix the named fields.

404 unknown application: check the identifier.

409 duplicate submission: fetch the recorded one.

Retry later

429 rate limited: wait for Retry-After.

503 dependency down: retry with backoff.

Timeout: outcome unknown, retry with the same key.

Every error body carries a type, a title and a traceId for support.

Choosing an integration style

Style	Fits when	LAS example
Direct request	The caller needs the answer now	Status query, credit check
Brokered events	Consumers complete work later	ApplicationRecorded fan-out
Batch and stream	Work is grouped or continuous	Nightly extracts, fraud windows

The nine patterns compose. Direct, brokered and grouped styles answer different needs.

ILLUSTRATIVE API CONTRACT

The submission request, field by field

Field	Illustrative value
method + path	POST /loan-applications
Idempotency-Key	9f2c-partner-417
customerId	C-2210
amount	25000.00 EUR
termMonths	60
response	201 + Location: /…/L-1042

Identity and meaning

The API assigns L-1042 exactly once.
The key makes retries identical.

Contract evolution

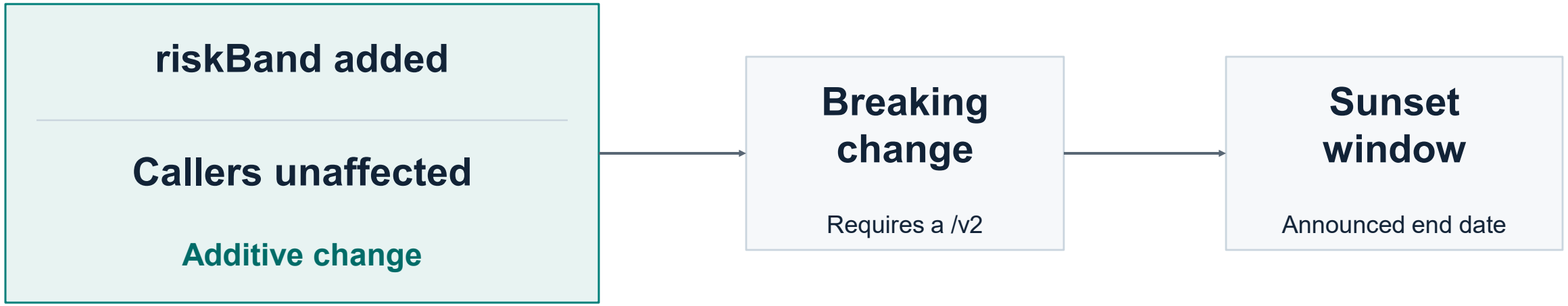
New fields arrive optional.
Meanings never change in place.

The gateway forwards the body untouched; the API validates every field.

COMPATIBLE CHANGE

Versioning without breaking callers

Contract v1

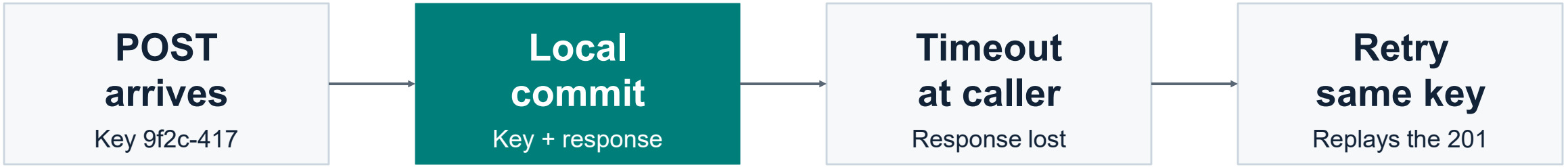


Run v1 and v2 side by side until monitored v1 usage reaches zero.

Additive change keeps v1 alive. A breaking change opens a parallel major version.

SAFE RETRIES

A partner retry, end to end



Same key: the stored response returns.
New key: a new application, by design.

A timeout hides whether the first call was recorded. The key resolves it.

Review question Which step makes the second POST safe?

Pagination, filtering and freshness

A list endpoint reads a dataset that keeps changing. Callers need stable rules for pages and filters.

Partners reconcile daily: GET /loan-applications?status=PendingAssessment.

Cursor pagination

Progress stays stable while rows are inserted.

Documented filters

Only status, submittedAt range and customerId.

Deterministic order

Sort by submittedAt, then id as tie-breaker.

Freshness bound

The list may lag writes; state the delay.

EDGE RESPONSIBILITIES

Gateway policies and business authority

At the gateway

Validate tokens and client identity.

Apply rate limits and payload caps.

Route versions and terminate TLS.

In domain services

Decide what this caller may do to L-1042.

Credit decisions stay in the Credit Decision Service.

Own record state and status transitions.

Passing the gateway is not permission to approve a loan.

PROTECTION CONTRACTS

Protecting the API in production

Failure or risk	Control	Evidence
Traffic spike	Partner quotas with 429 and Retry-After	Load test at twice the quota
Slow dependency	Deadlines and a circuit breaker	Status readable in the drill
Invalid payloads	Schema validation at the edge	Rejects logged with traceId

Each control needs an owner, a configured limit and evidence that it works before launch.

Operating a public contract

A published API is a product: its owners watch usage, test compatibility and manage change.

Follow one traceId from the partner call through gateway, API and credit check.

Contract tests

Consumers' expectations run in the pipeline.

Observability

Rate, errors and duration per operation.

Deprecation process

Sunset dates announced and usage tracked.

Single source

The OpenAPI document is the contract.

HYPOTHETICAL DECISION RECORD

Partner API decision

ADR-005 Partner submission API

Status: Proposed

Context

A partner portal needs to submit and track applications. Internal channels already use the pilot's APIs.

Decision

Publish one v1 contract behind the gateway: POST with a mandatory idempotency key, GET for status.

Consequences

Independent partner onboarding.

Key storage, quotas and monitoring. Breaking changes require a v2 and a sunset window.

GROUP EXERCISE

12 MINUTES

Exercise: the partner retry contract

Partner timeout scenario

POST /loan-applications for customer C-2210 times out.

The partner retries with the same idempotency key.

Your output

01 Choose status codes for both calls.

02 Define the error body for invalid data.

03 Set the retry and backoff rules.

04 State what each response proves.

HTTP 201 confirms recording. Pending assessment is not an approval.

EXERCISE DEBRIEF

Debrief: contracts that survive retries

01

One request, one record

The idempotency key turns the retry into the same request.

02

Replay, not re-execution

The retry receives the stored response, not a new effect.

03

Errors carry instructions

Each problem response tells the caller to fix, wait or stop.

04

Status is not approval

Recording, assessment and notification remain separate facts.

REFERENCE MATERIAL

References and further reading

HTTP semantics and error contracts

IETF HTTP and HTTPAPI working groups

- RFC 9110 HTTP Semantics RFC 9457 Problem Details

Idempotency and safe retries

IETF HTTPAPI working group and Stripe

- Idempotency-Key header draft Stripe idempotent requests

Choosing integration patterns

Gregor Hohpe and Microsoft Azure Architecture Center

- Request-Reply Asynchronous Request-Reply

Primary references. Topic-specific sources and teaching context appear in the speaker notes.